

コードの時系列変化を考慮した 保守性低下の要因分析と改善

鈴木研究室 2410064

笹川 尋翔

目次

1. 背景
2. 目的
3. 関連研究
4. 分析手順
5. ケーススタディ
6. 評価
7. 課題と展望

1. 背景

76%の開発者がリファクタリングによるバグ混入のリスクを認識 [1]

モジュール内部や外部の構造が変化し、静的分析では品質改善が困難に

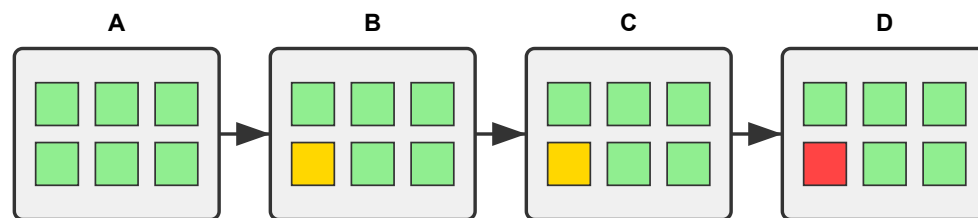


図1 欠陥の混入と顕在化

[1] Microsoft Research, "An Empirical Study of Refactoring Challenges and Benefits at Microsoft," 2014

2. 目的

- ソフトウェアメトリクスの変化的変化を通じてバグ混入リスクを事前に特定
- 保守性の低下傾向を早期に検出し、バグが発生する前に対策を講じるための情報を提供

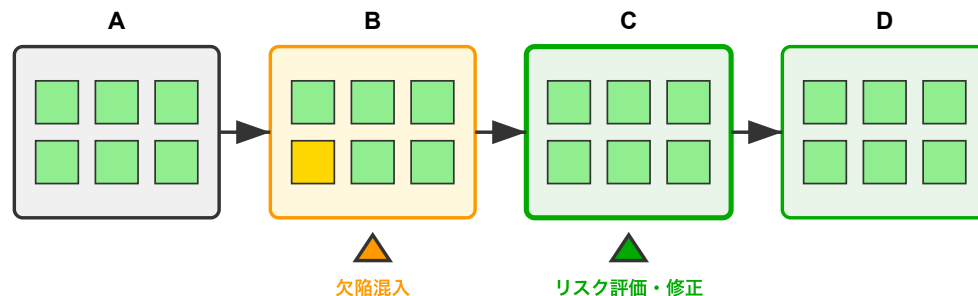


図2 欠陥の特定と修正

3. 関連研究

- Hanらは、レビューの分析により欠陥の種類を調査したが、レビューのうち70%では明示的に欠陥が指摘されなかった[2]
- Romanoらは、静的なしきい値に基づいた警告が欠陥修正作業を効率化することを示したが、動的なしきい値については検証していない[3]

[2] X.Han et al., "Understanding Code Smell Detection via Code Review: A Study of the OpenStack Community," 2021

[3] S.Romano et al., "Do Static Analysis Tools Affect Software Quality when Using Test-driven Development?," 2022

3. 関連研究

- Ferencらは、コミットごとのソフトウェアメトリクスを用いて欠陥予測を実施したが、メトリクスの変化量を導入しなかった^[4]
- Kameiらは、コミットデータに基づく14の変更メトリクスを提案しているが、ソフトウェアメトリクスとの関連付けが不十分^[5]

[4] R.Ferenc et al., "An automatically created novel bug dataset and its validation in bug prediction," 2020

[5] Y. Kamei et al., "A large-scale empirical study of just-in-time quality assurance," 2013

4. 分析手順

1. データセットを構築
2. ソフトウェアメトリクスの変化量を追加
3. 特徴量の追加前・追加後のデータを用いて機械学習モデルを訓練
4. 各プロジェクトでモデルの性能を評価



図3 分析手順

4.1 データセットの構築

- Ferencらが作成したBugHunter Dataset^[4]を使用
 - コミットごと、ソフトウェアの粒度（例: クラス、メソッド）ごとのソフトウェアメトリクスが含まれる
- モデルの精度向上のために前処理を行った
 - 欠損値を削除し、テキストデータをベクトル化

4.2 メトリクスの変化量の追加

- 比較的単純な特徴量として、コード行数・トークン数・循環的複雑度の変化量を追加
- 直前のコミットとの差分を用いて変化量を計算

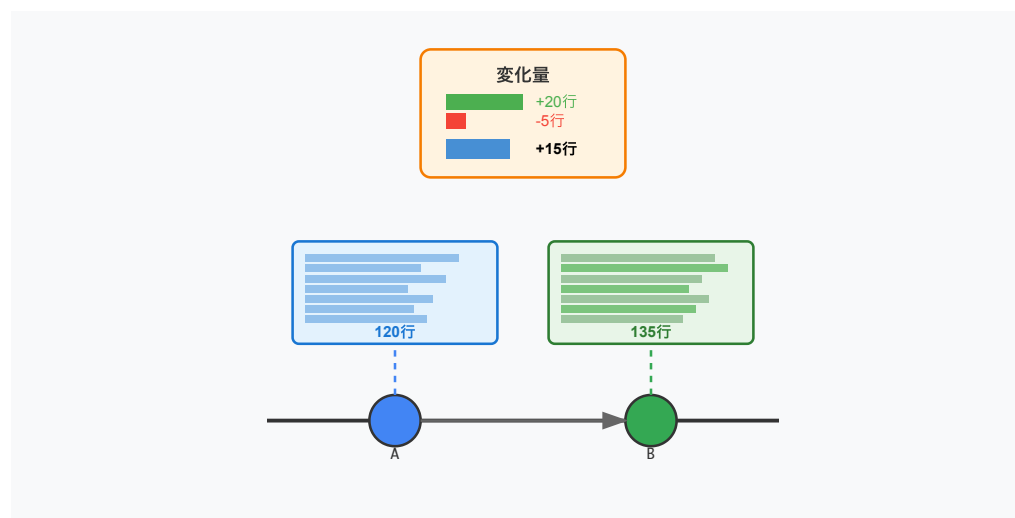


図4 変化量の計算方法

4.3 機械学習モデルの訓練

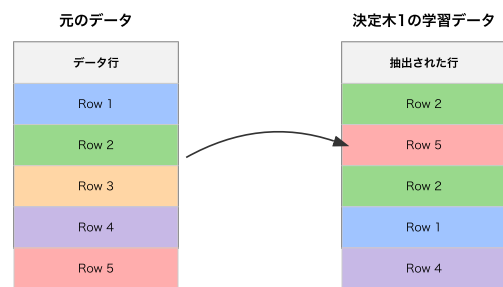


図5 ブートストラップ
法

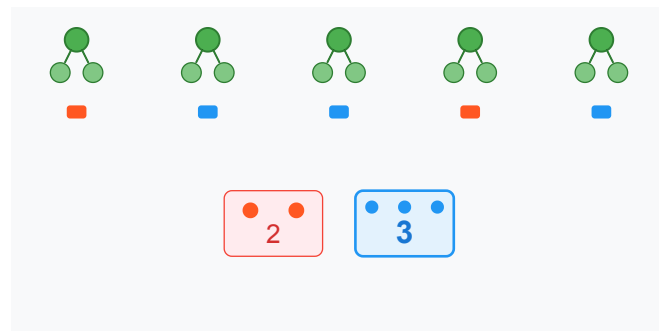


図6 決定木を用いた多
数決

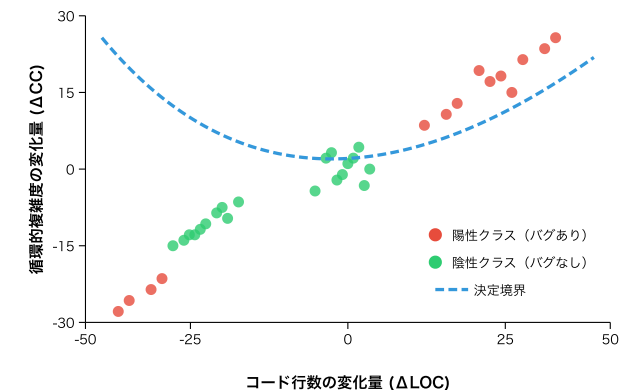


図7 決定境界による分
類

4.4 プロジェクト内評価

- Zhaoらによれば、欠陥予測ではF1スコアによる評価が一般的^[6]
- この研究でも同様に、F1スコアによる評価を行う
- 評価の確信度を測るために有意性検定も実施

5. ケーススタディ

- プロジェクト内の適用可能性を調べるために、BugHunter Datasetから6プロジェクトを選択し、欠陥の有無を予測

5.1 分析手法

1. 特徴量重要度（Feature Importance）を算出
2. ヒストグラムで特徴量分布を確認
3. Partial Dependence Plot（PDP）を用いて分類傾向を把握
4. 決定木で分類の流れを可視化し、判断の基準と確信度を確認

5.2 プロジェクト内分析

表1 選定したプロジェクト

プロジェクト	コード行数 (概算)	コミット数	バグレポートの数
Elasticsearch	995,000	28,815	4,494
Hazelcast	949,000	24,380	3,882
Broadleaf Commerce	322,000	14,920	703
Eclipse plugin for Ceylon	181,000	7,984	923
ANTLR v4	68,000	6,526	179
Oryx	34,000	1,054	67

5.3 特徴量重要度

- トークン数・コード行数の変化量、Halsteadメトリクス、Maintainability Indexの重要度が高い

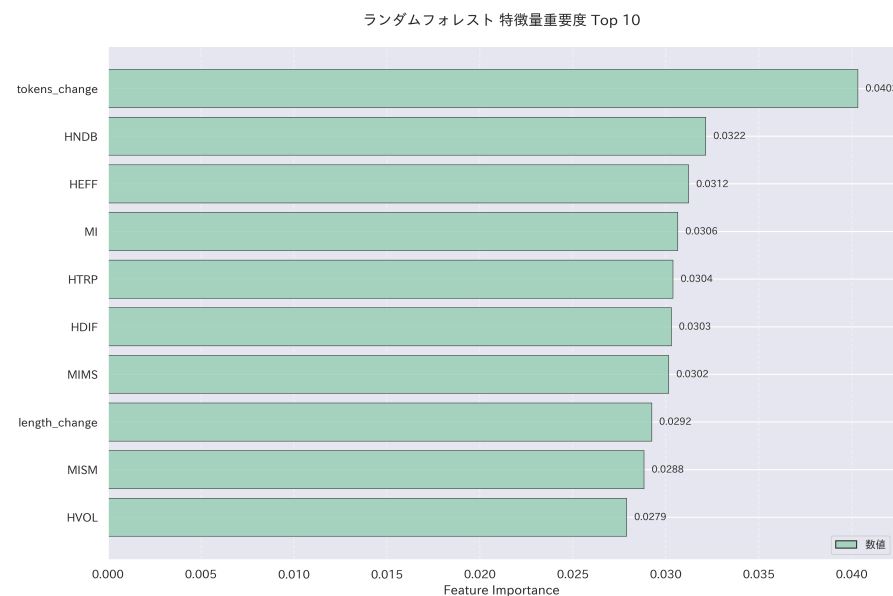


図8 代表的な特徴量重要度

5.4 特徴量分布

- 変化量やHalstead系は分散が小さく、Maintainability Indexは分散が大きい

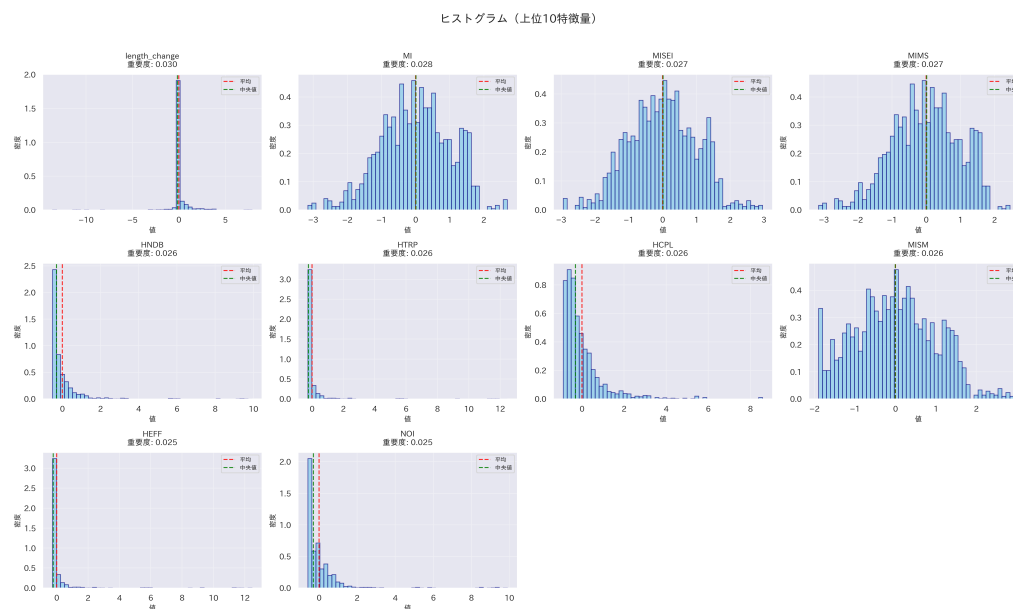


図9 代表的な特徴量分布

5.5 PDP分析

- ほとんどの特徴量において、バグありの予測確率が0.5未満

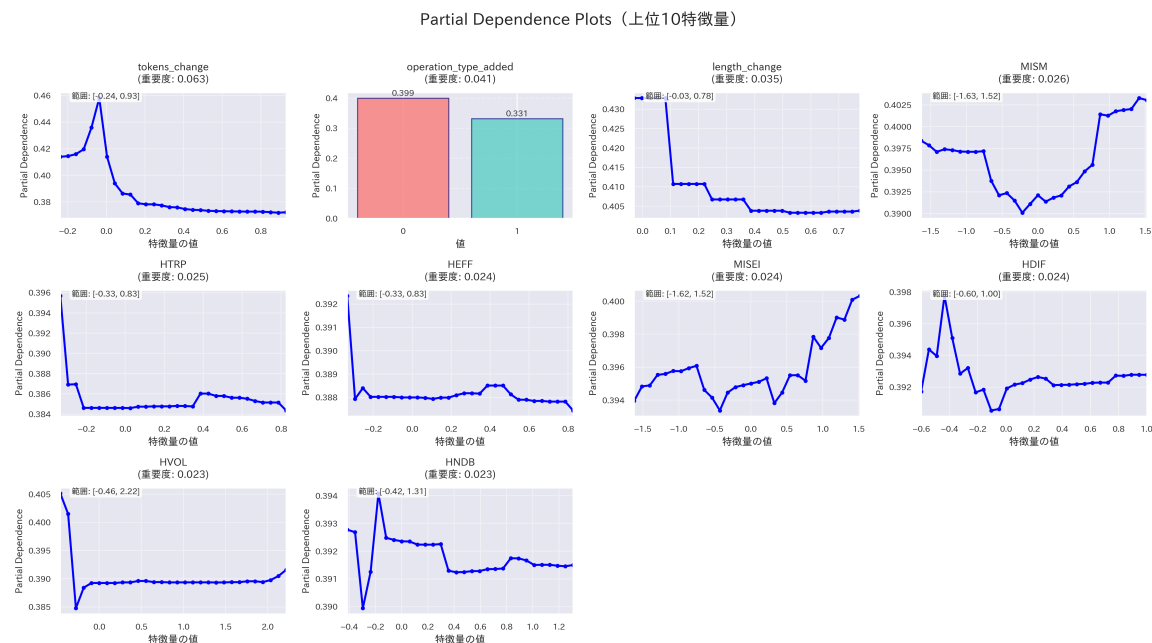


図10 代表的なPDP

5.6 決定木分析

- バグなしと判断したときの確信度が比較的高い

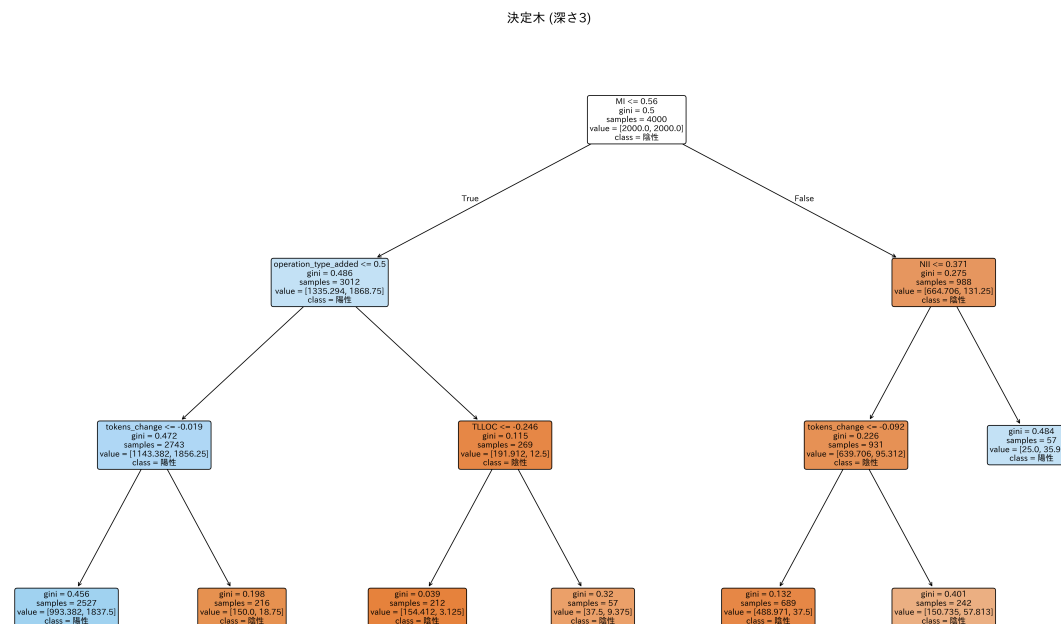


図11 代表的な決定木

5.7 評価指標の測定

表2 評価指標の値の変化

プロジェクト	F1スコア (変更前)	F1スコア (変更後)
Eclipse plugin for Ceylon	0.39	0.49
Elasticsearch	0.62	0.72
Hazelcast	0.67	0.71
ANTLR v4	0.45	0.48
Oryx	0.33	0.39

モデルの予測性能が改善される傾向があることを確認

6. 評価

- 達成できたこと
 - ソフトウェアメトリクスのコミット間の変化量を活用することでより精度の高い分類ができることを示した
- 達成できなかったこと
 - 欠陥混入の根本的な原因である開発プロセスについて分析していない

7. 課題と展望

- 開発プロセスを定量的に分析
 - VCS (Version Control System) を活用し、開発プロセス関連の情報を得る
- 欠陥予測の確信度を改善
 - タスクの優先順位付けを行いやすくし、リスクに基づいた意思決定を支援